

END-OF-STUDY PROJECT REPORT

Submitted in Partial Fulfillment of the Requirements for the
BACHELOR DEGREE IN COMPUTER SCIENCE

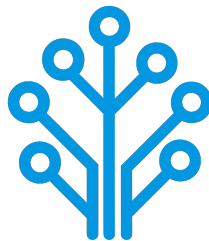
Field of Study : Software Engineering and Information Systems

Design and Implementation of an AI-Based Network Intrusion Detection System

By

AMIRA BOUAFIF & HAJER JEMAA

Conducted within ITXTREE



Publicly defended on June 6, 2026 in front of the jury members:

Chairman:	Name SURNAME, University Relations Leader, IBM
Reporter:	Name SURNAME, Teacher, ISTIC
Examiner:	Name SURNAME, Teacher, ISTIC
Professional Supervisor:	Firas ABBESSI, Engineer, ITXTREE
Academic Supervisor:	Wassim ABBESSI, Teacher, ISTIC

Academic Year: 2025-2026

END-OF-STUDY PROJECT REPORT

Submitted in Partial Fulfillment of the Requirements for the
BACHELOR DEGREE IN COMPUTER SCIENCE

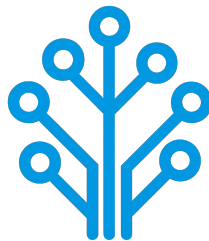
Field of Study : Software Engineering and Information Systems

Design and Implementation of an AI-Based Network Intrusion Detection System

By

AMIRA BOUAFIF & HAJER JEMAA

Conducted within ITXTREE



Authorization of graduation project report submission:

Professional Supervisor:

Academic Supervisor:

Issued on :

Issued on :

Signature:

Signature:

Dedications

I dedicate this End-of-Study project

To my dear parents

Who have supported and encouraged me every step of the way. This is for you, in honor of all the sacrifices you have made for me. I hope to have made you both proud and happy.

To my dear sisters and brothers

Who have been a great support and a source of joy in my life. I am grateful for your love and encouragement throughout this journey.

To all my family and friends

Thank you all for your support and encouragement throughout all this time. I am truly blessed to have you in my life.

Amira BOUAFIF

Dedications

I dedicate this End-of-Study project

To my dear parents

To my dear sisters

To all my family and friends

Hajer JEMAA

Acknowledgments

We would like to extend our sincere gratitude to all those who have supported us throughout the course of this End-of-Study project. Their guidance, encouragement, and support have been invaluable throughout this journey.

First and foremost, we would like to thank our academic supervisor, Professor Wassim ABBESSI, for his guidance and insights, which have been a great help in the successful completion of this project.

Our gratitude also go to ITXTREE and, in particular, our professional supervisor, Mr. Firas ABBESSI, for providing us with the opportunity to work on this project and for their professional mentorship.

We also extend our deep appreciation to the professors at the Higher Institute of Information and Communication Technologies for their invaluable support and teachings, which have greatly contributed to our academic journey.

Finally, we thank everyone who has, in one way or another, helped in making this project a success.

Contents

Dedications	i
Dedications	ii
Acknowledgments	iii
General Introduction	1
1 Project Context	2
1.1 Introduction	2
1.2 Presentation of the company	2
1.3 Study of the existing	2
1.4 Criticism of the existing	3
1.5 Proposed solution	3
1.6 Project Pipeline	4
1.7 Methodologies	4
1.7.1 CRISP-DM	4
1.7.2 UML	5
1.8 Conclusion	6
2 Requirements Specification	7
2.1 Introduction	7
2.2 Functional Requirements	7
2.3 Non-Functional Requirements	7
2.4 Actors Identification	8
2.5 Use Case Diagram	9
2.6 Conclusion	9

3	Conceptual Design	11
3.1	Introduction	11
3.2	Sequence Diagrams	11
3.3	Class Diagram	11
3.4	Deployment Diagram	11
3.5	Conclusion	11
4	Data Collection and Preparation	12
4.1	Introduction	12
4.2	Data Collection	12
4.2.1	Dataset comparative	13
4.2.2	Dataset overview	14
4.3	Data Understanding	14
4.3.1	Initial Exploration	14
4.3.2	Data Description	14
4.3.3	Data Exploration	21
4.3.4	Data quality	26
4.4	Data preparatation	32
4.4.1	Data cleaning	32
4.4.2	Feature engineering	34
4.4.3	Label encoding	36
4.4.4	Train/ Test split	36
4.5	Conclusion	37
5	Modeling	38
5.1	Introduction	38
5.2	State of the Art	38
5.3	Adopted Approach	38
5.4	Training and Hyperparameter Optimization	38
5.5	Conclusion	38
6	Realization	39
6.1	Introduction	39
6.2	Development Tools and Technologies	39

6.3	Performance Analysis and Evaluation Metrics	39
6.4	User Interfaces	39
6.5	Integration	39
6.6	Conclusion	39
General Conclusion		40

List of Figures

1.1	Project Pipeline	4
1.2	CRISP-DM Methodology	5
2.1	Actors	8
2.2	Use Case Diagram	10
4.1	Label Corruption	15
4.2	BENIGN vs Attack Ratio	21
4.3	Benign vs Attack Ratio per file	22
4.4	Class distribution per file	22
4.5	Attack distribution	23
4.6	Correlation Heatmap	25
4.7	Inf and NaN attack class distribution	28
4.8	Ordering Artifact Class Distribution	29
4.9	Exact duplicates attack class distribution	30
4.10	ontradictory duplicates attack class distribution	31

List of Tables

2.1	Roles of Actors	9
4.1	Comparison of network intrusion detection datasets	13
4.2	Dataset Files	14
4.3	Feature Groups	15
4.4	Feature Statistics	17
4.5	Class Distribution Per File	17
4.6	Attack Classes	18
4.7	Class Distribution	21
4.8	Perfectly Correlated Features	25
4.9	Domain Knowledge decision	35

General Introduction

Chapter 1

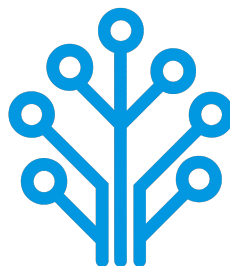
Project Context

1.1 Introduction

This chapter serves as an introduction to the project. We begin with a presentation of the company hosting the internship, a study of existing network intrusion detection systems. Then, we present the problem and the proposed solution. Finally, we end the chapter with a conclusion.

1.2 Presentation of the company

ITXTREE is an early-stage technology startup based in Sidi Rzig, Tunisia, focused on building a solid foundation for future digital solutions it is built on the core values of integrity, quality, curiosity and adaptability.



1.3 Study of the existing

Cybersecurity is a highly sensitive domain where the ability to react quickly can determine the impact of an attack. The earlier a threat is identified, the more effectively its consequences can be limited.

For this reason, Network Intrusion Detection Systems (NIDS) play a central role in monitoring and analyzing network traffic. The most widely used approach relies on:

- **Signature-based:** These systems analyze network traffic, often through deep packet inspection and compares it against a signature database. When a match is found an alert is triggered. Well-known tools such as Snort [1] and Suricata [2] are representative of this approach.
- **Anomaly-based:** Anomaly detection is learning normal behavior of the network through statistical analysis of traffic parameters. If there is a sudden increase of traffic on a port outside of normal thresholds, it might be an indication of a brand-new threat [1].

1.4 Criticism of the existing

Despite their widespread use, signature-based NIDS present several limitations:

- **High Maintenance and Reactivity:** Their effectiveness is essentially tied to the completeness and accuracy of their signature rule sets. As attack techniques continuously evolve, these systems require frequent updates to remain effective. This maintenance process is both time-consuming and reactive since new signatures can only be created after an attack has already been identified and analyzed.
- **Performance:** As the number of rules increases, the system must process a larger set of comparisons for each packet, which can lead to slower detection. In a context where reaction time is critical, this degradation directly affects the system's ability to respond effectively.
- **Rigidity and Evasion Vulnerability:** Attackers can exploit the rigidity of signature-based detection systems by introducing slight variations in attack patterns, allowing them to evade existing signatures while preserving the same malicious intent. Since many signature rules are publicly available or widely shared with the security community, attackers may analyze them to identify gaps and design variants that bypass detection mechanisms.

Overall, these systems rely on exact matching and predefined logics which limits their adaptability in the face of evolving and increasingly sophisticated threats.

1.5 Proposed solution

With the growing integration of Artificial Intelligence (AI) in security systems, new approaches aim to overcome the limits of traditional methods. To address these challenges, we propose an AI-driven NIDS capable of learning patterns from network traffic and classifying different types of network activity instead of relying solely on predefined rules.

1.6 Project Pipeline

A project pipeline is a structured sequence of stages that outlines the workflow and processes involved in the development and deployment of a project. It serves as a roadmap, guiding the team through various phases, from initial planning to final delivery.

Figure 1.1 illustrates the project pipeline for our network intrusion detection system.

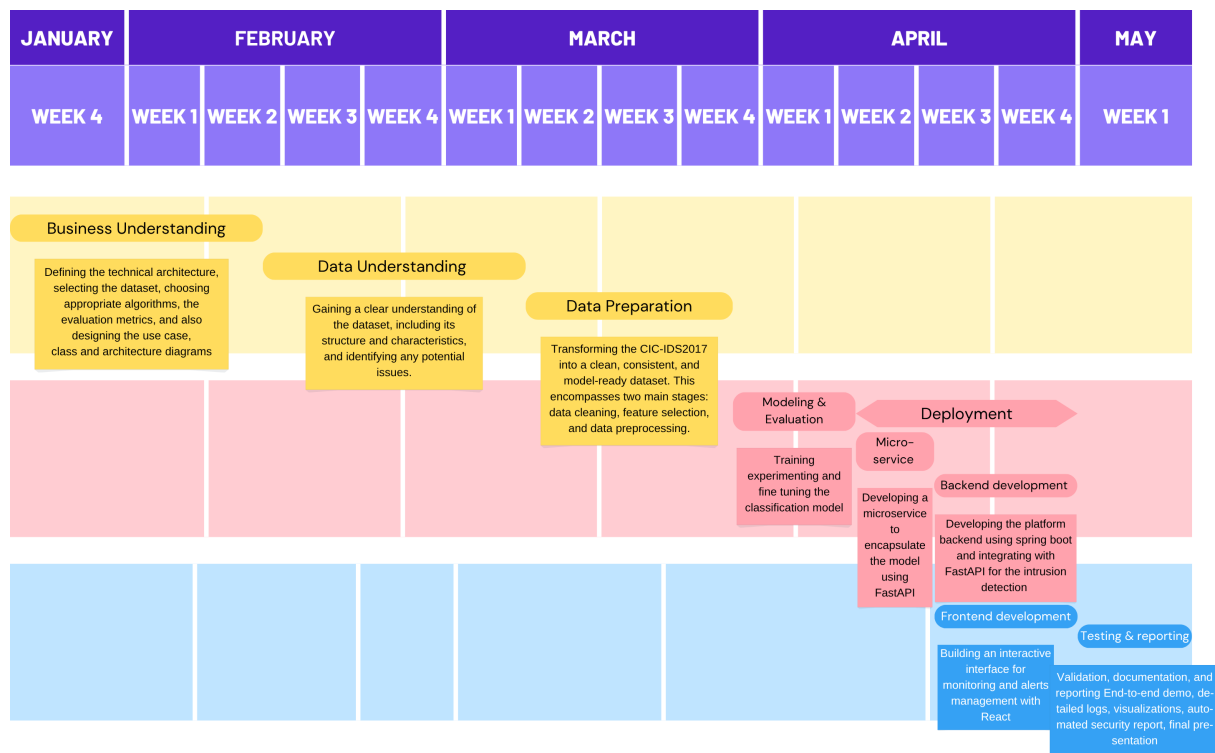


FIGURE 1.1 – Project Pipeline

1.7 Methodologies

In this section, we present the methodologies that we use in this project, including the machine learning methodology CRISP-DM and the software development methodology UML.

1.7.1 CRISP-DM

The CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology is a widely used framework for structuring machine learning projects. It consists of six phases:

- **Business Understanding:** Defining project objectives and requirements and gaining a clear understanding of the business problem.

- **Data Understanding:** Collecting and exploring the data to gain insights and assess its quality by identifying data quality issues.
- **Data Preparation:** Cleaning and preparing the data for modeling by selecting relevant features, handling missing values and splitting the dataset.
- **Modeling:** Selecting and applying appropriate modeling techniques, building, testing and tuning models to optimize their performance.
- **Evaluation:** Evaluating the model's performance and determining if it meets the business objectives.
- **Deployment:** Deploying the model into production, monitoring and maintaining the model's performance.

Figure 1.2 illustrates these phases and their relationships.

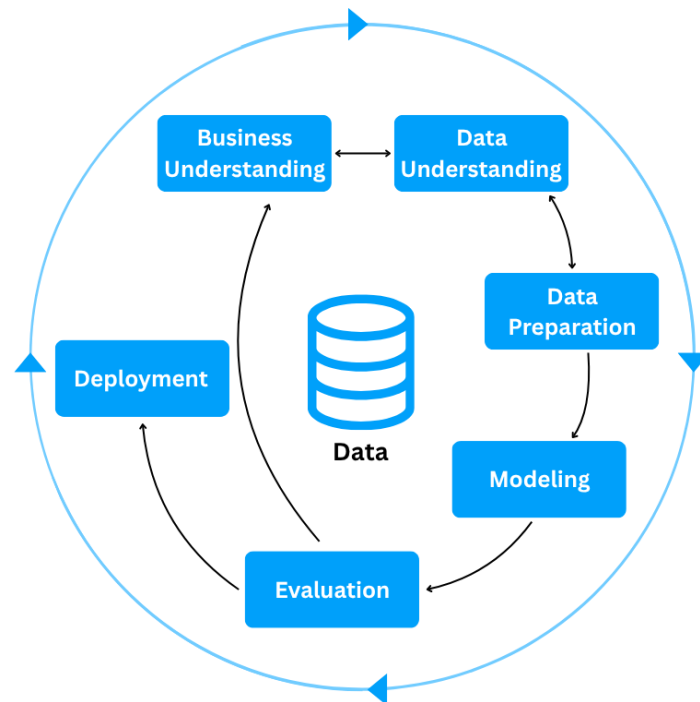


FIGURE 1.2 – CRISP-DM Methodology

1.7.2 UML

For the modeling of the software system, we use UML (Unified Modeling Language), a standardized modeling language used to specify, visualize, construct, and document the artifacts of software systems through diagrams such as use case, class, and sequence diagrams. There are two types of UML diagrams:

- **Behavioral Diagrams:** State machine, activity, use case, sequence, communica-

tion and interaction overview diagrams.

- **Structural Diagrams:** Class, object, component, deployment and package diagrams.

UML makes complex systems easier to design and communicate, it provides a common language for developers, architects, and stakeholders [3].

1.8 Conclusion

In this chapter, we introduce the host company ITXTREE, study existing network intrusion detection systems, discuss their shortcomings and finally present a solution. In the next chapter we specify the system requirements.

Chapter 2

Requirements Specification

2.1 Introduction

Throughout this chapter, we specify both the functional and non-functional requirements, identify the system actors, and present the use case diagram. By defining these elements, we aim to establish a clear and shared understanding of the system's expected behavior and constraints, providing a solid foundation for the subsequent design and implementation phases.

2.2 Functional Requirements

Functional requirements define *what* the system should perform. The main functionalities of our system are as follows:

- Authenticate;
- View Dashboard;
- View Statistics summary;
- Trigger Network Analysis;
- View Alerts List;
- View Reports;

2.3 Non-Functional Requirements

Non-functional requirements define *how* the system should perform its functions. They describe the quality attributes and constraints of the system:

- **Performance:** The system must process network traffic and generate analysis results with minimal latency, ensuring fast detection of potential intrusions.
- **Security:** The system must ensure secure authentication and enforce role-based access control, preventing unauthorized actions and protecting system integrity, while supporting non-repudiation to guarantee accountability and traceability of all operations.
- **Maintainability:** The system should be easy to maintain, allowing for efficient debugging, updates, and modifications to existing components.
- **Usability:** The system should provide an intuitive interface, enabling users to efficiently navigate and interact with its functionalities.

2.4 Actors Identification

Actors are external entities, users, organizations, or external systems that interact with the system to achieve one or more functional requirements, with each actor representing a specific role.

In the context of our project, we identify two primary actors, as illustrated in Figure 2.1

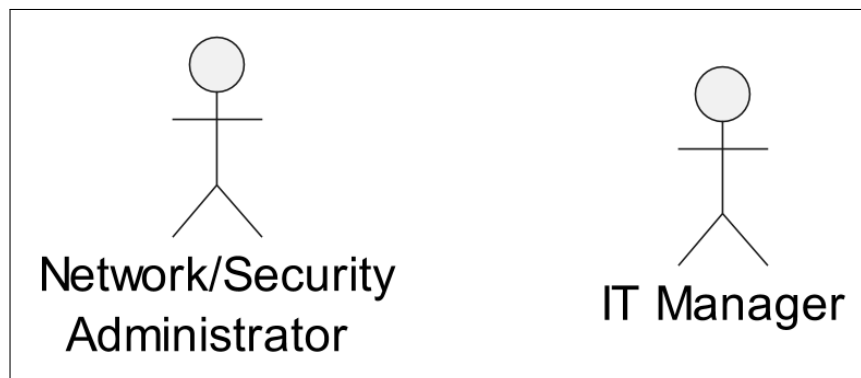


FIGURE 2.1 – Actors

Table 2.1 presents the roles of each actor in our system.

TABLE 2.1 – Roles of Actors

Actor	Role
Network/Security Administrator	- Authenticate - View Dashboard - View Statistics Summary - Trigger Network Analysis - View Alerts List
It Manager	- Authenticate - View Dashboard - View Statistics Summary - View Reports

2.5 Use Case Diagram

A use case diagram is a visual representation of *how* users interact with the system and *what* a system does through different use cases. Figure 2.2 illustrates that diagram.

2.6 Conclusion

In this chapter, we outline the functional and non-functional requirements of our system, identify the primary actors, and represent the way they interact with the system and the system's functionality via a use case diagram. In the next chapter, we present the conceptual design of the platform.



FIGURE 2.2 – Use Case Diagram

Chapter 3

Conceptual Design

3.1 Introduction

3.2 Sequence Diagrams

3.3 Class Diagram

3.4 Deployment Diagram

3.5 Conclusion

Chapter 4

Data Collection and Preparation

4.1 Introduction

4.2 Data Collection

This section introduces the process of acquiring the data used in this project. In general, this phase may involve either constructing a dataset from scratch or relying on existing sources. In our case, the decision to use a public dataset is mainly dictated by the nature of the cybersecurity domain.

Unlike other fields, network intrusion data cannot be collected on demand, as it depends on the occurrence of unpredictable and potentially harmful attacks. In addition, data collection in real operational environments is often constrained due to privacy concerns, security risks, and limited access to organizational traffic. Furthermore, even when such data is available, obtaining reliable ground truth labels is difficult and typically requires extensive manual analysis.

For these reasons, constructing a proprietary dataset is impractical, particularly in the absence of real network traffic provided by the host organization. Consequently, the use of a public dataset becomes a necessary and coherent choice, as such datasets are generated in controlled environments and include labeled instances based on expert-defined attack scenarios.

4.2.1 Dataset comparative

We narrowed it down to these three datasets. Below is a comparative table 4.1 using our primary selection criteria.

TABLE 4.1 – Comparison of network intrusion detection datasets

Dataset	Source	Size	Data Generation Approach	Attack Diversity and Temporal Relevance	Usability
NSL-KDD [NSL-KDD]	Canadian Institute for Cybersecurity at the University of New Brunswick (UNB)	Small ($\approx 148k$ records)	Derived from the KDD '99 dataset, its traffic was generated in a military-style network environment with predefined automated attack scripts.	Low; 4 attack families: DoS, Probe, R2L, U2R with several specific attacks in each family.	High; legacy benchmark widely used in academic studies, mainly for baseline comparison and educational purposes.
UNSW-NB15 [UNSW-NB15]	Australian Centre for Cyber Security (ACCS), UNSW Canberra	Medium ($\approx 2.5M$ records)	Synthetic traffic generated with IXIA PerfectStorm tool.	Medium; 9 attack types: Fuzzers, Analysis, Backdoor, DoS, Exploits, Generic, Reconnaissance, Shellcode, Worms.	High; widely used modern benchmark for machine learning-based intrusion detection research.
CIC-IDS2017 [CIC-IDS2017]	Canadian Institute for Cybersecurity at the University of New Brunswick (UNB)	Medium ($\approx 2.8M$ records)	Traffic generated in an enterprise-like network environment composed of attacker and victim machines, using real attack tools.	High; 14 attack types: Includes the most common attacks based on the 2016 McAfee report, such as Web based, Brute force, DoS, DDoS, Infiltration, Heart-bleed, Bot and Scan.	Very High; one of the most widely adopted and current standard datasets in intrusion detection research.

The comparative analysis of the three datasets reveal that each has its own limitations. NSL-KDD, although still relevant, exhibits limitations in size and temporal relevance, as it is relatively old and does not reflect recent network behavior and modern attacks types.

UNSW-NB15 is larger and offers a more recent alternative, but it remains limited in terms of attack type coverage, and its traffic is fully synthetically generated. This makes CIC-IDS2017 the most suitable dataset, as it provides a more realistic experimental setting, a wider range of the most common attack types, and it is also widely adopted in recent research studies.

4.2.2 Dataset overview

CIC-IDS2017 was created by the Canadian Institute for Cybersecurity (CIC) at the University of New Brunswick (UNB), within a controlled environment, where the network traffic was captured in packet capture (PCAP) format.

The transformation of raw PCAP files was done using a feature extraction tool, CICFlowMeter. Therefore a row in this dataset doesn't represent a packet, it represents an overview of the session behaviour between two endpoints through statistical summary.

4.3 Data Understanding

The Data Understanding phase of the CRISP-DM methodology focuses on loading, describing, and analyzing the dataset to build a solid foundation for the rest of the project. This section dives into the four tasks of this phase: **Initial Exploration**, **Data Description**, **Data Exploration**, and **Data Quality**, with each task targeting an aspect of the data, from its structure and content to its quality.

4.3.1 Initial Exploration

The CIC-IDS2017 dataset is captured over five days, it is distributed across 8 files split by day and session: **Monday**, **Tuesday**, and **Wednesday** each corresponding to a single file, while **Thursday** and **Friday** are split into AM and PM sessions. The **Friday PM** session is further divided into two files to isolate the two attacks, **DDoS** and **PortScan**.

4.3.2 Data Description

As mentioned in the previous subsection, the dataset is split into 8 files as shown in Table 4.2.

TABLE 4.2 – Dataset Files

File	Number of Rows	Number of Columns
Monday	529,918	79
Tuesday	445,909	79
Wednesday	692,703	79
Thursday AM	170,366	79
Thursday PM	288,602	79
Friday AM	191,033	79
Friday PM (DDoS)	225,745	79
Friday PM (PortScan)	286,467	79
Total	2,830,743	—

The dataset contains a total of 2,830,743 rows across the 8 files. All files have consistent columns, which is expected as they are generated by the same CICFlowMeter pipeline. This consistency allows for seamless concatenation of the files into a single DataFrame for the subsequent analysis.

- **Whitespace Issue:** Leading white spaces is found in columns names and are stripped across all files as they can raise `KeyError` exceptions when accessing columns by name.
- **Corrupted Labels:** Corrupted labels are discovered in the Thursday AM file, where three labels contain mojibake character instead of an em-dash (–) caused by an encoding issue as shown in Figure 4.1 and are therefore corrected.

```
[Thursday-AM] 'Web Attack ⬠ Brute Force'
[Thursday-AM] 'Web Attack ⬠ XSS'
[Thursday-AM] 'Web Attack ⬠ Sql Injection'
```

FIGURE 4.1 – Label Corruption

These fixes are purely cosmetic and do not alter the underlying data values.

- **Feature Description:** The dataset contains 79 columns, of them 78 are numeric level-flow features and the last one is the target Label. These features can be grouped into 12 categories as shown in Table 4.3.

TABLE 4.3 – Feature Groups

Group	Features	Description
Flow Metadata	Destination Port, Flow Duration	Total duration of the flow and the destination port
Packet/Byte Count	Total Fwd Packets, Total Backward Packets, Total Length of Fwd Packets, Total Length of Bwd Packets, act_data_pkt_fwd	Number of packets and packet bytes in both forward (from source to destination) and backward directions (from destination to source), and packets having actual data in the forward direction.
Packet/Segment Length Statistics	Fwd Packet Length Max/Min/Mean/Std, Bwd Packet Length Max/Min/Mean/Std, Max/Min Packet Length, Packet Length Mean/Std/Variance, Average Packet Size, Avg Fwd Segment Size, Avg Bwd Segment Size	Packets and segments length statistics (e.g. max, min, mean, std etc...).

Flow Rate	Flow Bytes/s, Flow Packets/s, Fwd Packets/s, Bwd Packets/s, Down/Up Ratio	Flow throughput, forward and backward packet rates, flow asymmetry ratio (backward packets (download) to forward packets (upload)).
Inter-Arrival Time	Flow IAT Max/Min/Mean/Std, Fwd IAT Max/Min/Mean/Std/Total, Bwd IAT Max/Min/Mean/Std/Total	Time between consecutive packets in the flow, forward, and backward directions.
TCP Flag Count	FIN Flag Count, SYN Flag Count, RST Flag Count, PSH Flag Count, ACK Flag Count, URG Flag Count, CWE Flag Count, ECE Flag Count, Fwd PSH Flags, Bwd PSH Flags, Fwd URG Flags, Bwd URG Flags	TCP flags counts for the entire flow, forward and backward directions.
Header Length	Fwd Header Length, Bwd Header Length, Fwd Header Length.1	Backward and forward header lengths.
Bulk Transfer Statistics	Fwd Avg Bytes/Bulk, Fwd Avg Packets/Bulk, Fwd Avg Bulk Rate, Bwd Avg Bytes/Bulk, Bwd Avg Packets/Bulk, Bwd Avg Bulk Rate	Average forward and backward bulk (a sequence of consecutive packets in the same direction) transfer statistics.
Subflow Packet/Byte Count	Subflow Fwd Packets, Subflow Fwd Bytes, Subflow Bwd Packets, Subflow Bwd Bytes	Number of packets and packet bytes in both directions in a subflow.
Window/Segment Size	Init_Win_bytes_forward, Init_Win_bytes_backward, min_seg_size_forward	Window sizes in both the forward and backward directions and the minimal segment size in the forward direction.
Active/Idle Time	Active Max/Min/Mean/Std, Idle Max/Min/Mean/Std	Duration of active and idle periods within a flow.
Target	Label	The target label indicating the class the flow belongs to.

- **Duplicate Column:** The column `Fwd Header Length.1` is identified as a duplicate of `Fwd Header Length`, confirmed by the fact that both contain the same values.
- **Packet Length Mean and Average Packet Size** both measure the average packet size, but the two do not hold the same values, and they are calculated differently: the former uses the statistical mean, while the latter divides the total length of packets by the total number of packets[4].

All 54 features are of type `int64`, 24 features are of type `float64`, with the last feature, the target *Label*, being of type `object`.

Table 4.4 presents the statistics of some of the features.

TABLE 4.4 – Feature Statistics

Feature	Min	Max	Mean	Std
Flow Duration	-13	119,999,998	14,785,663.92	33,653,744.09
Total Backward Packets	0	121,922	10.39	997.39
act_data_pkt fwd	0	213,557	5.42	636.43
Min Packet Length	0	1,448	16.43	25.24
Flow Bytes/s	-241,000,000	inf	inf	nan
Flow IAT Max	-13	120,000,000	9,182,475.32	24,459,539.25
Bwd PSH Flags	0	0	0	0
Fwd Header Length	-322,212,234,632	4,644,908	-25,997.39	21,052,857.87
Bwd Avg Bulk Rate	0	0	0	0

Some of the feature has negative, inf and constant statistic of zero. Those features are further investigated in the data quality section.

- **Class Distribution:** The dataset contains 15 classes, BENIGN and 14 attacks, Table 4.5 shows the distribution of these classes across the dataset files.

TABLE 4.5 – Class Distribution Per File

File	Class	Class Count
Monday	BENIGN	529,918
Tuesday	BENIGN	432,074
	FTP-Patator	7,938
	SSH-Patator	5,897
Wednesday	BENIGN	440,031
	DoS Hulk	232,443
	DoS GoldenEye	10,293
	DoS Slowloris	5,796
	DoS Slowhttptest	5,499
	Heartbleed	11
Thursday AM	BENIGN	168,186
	Web Attack - Brute Force	1,507
	Web Attack - XSS	652
	Web Attack - Sql Injection	21
Thursday PM	BENIGN	288,566
	Infiltration	36
Friday AM	BENIGN	189,067
	Bot	1,966
Friday PM (DDoS)	DDoS	128,027
	BENIGN	97,718
Friday PM (PortScan)	PortScan	158,930
	BENIGN	127,537

With the exception of Monday, which contains only BENIGN traffic, all files contain at least one attack class alongside BENIGN samples. Each attack class is present in only one file.

Table 4.6 provides a description of the 14 attack classes of the dataset.

TABLE 4.6 – Attack Classes

Class	Description
FTP-Patator (File Transfer Protocol Patator)	A brute force attack using the Patator tool targeting FTP servers, an old unencrypted protocol that transmits data and credentials in plain text, using trial and error to guess credentials in order to gain unauthorized access.
SSH-Patator (Secure Shell Patator)	A brute force attack using the Patator tool targeting SSH servers, a secure encrypted protocol that encrypts the entire session including the login, using trial and error to gain unauthorized access to individual accounts.
DoS Hulk	A Denial-of-Service attack using the HULK tool, which is used to overwhelm web servers by generating a large number of unique randomized requests at irregular intervals.
DoS GoldenEye	A Denial-of-Service attack using the GoldenEye tool, which dynamically generates multiple parallel connections (multi-threaded) to a target URL while keeping them alive to exhaust the available connection slots, simultaneously forcing the server to bypass caching mechanisms and fully process every request.
DoS slowloris	A Denial-of-Service attack using the Slowloris tool, which enables a single machine to overwhelm a web server by opening multiple connections and keeping them open by sending partial requests (only headers) periodically, while requiring very low bandwidth, hence its name, as it operates in a low and slow manner.
DoS Slowhttpstest	A Denial-of-Service attack using the SlowHTTPTest tool, which creates very slow connections by sending seemingly legitimate requests using low bandwidth and intentionally delaying their completion, thereby consuming the limited number of concurrent connections the server can handle.

Heartbleed	An attack in which the attacker exploits the Heartbleed bug in the OpenSSL cryptographic library to read sensitive memory contents by sending malformed heartbeat requests demanding more memory than the actual payload from the server running the vulnerable version of OpenSSL, after which the server allocates the requested memory without any verification and returns it, exposing whatever sensitive data happened to be stored in that memory chunk at that moment. It is used to steal private encryption keys, usernames and passwords, and to intercept session tokens, permitting session hijacking by providing the stolen token to the server so that the attacker is treated as a legitimate logged-in user, as well as to decrypt previously captured encrypted traffic that the attacker had intercepted beforehand.
Web Attack - Brute Force	An attack that uses a forceful approach to steal credentials to gain unauthorized access to systems. It can be carried out in many ways, from trying common credential combinations manually, to using dictionary attacks, or a combination of both.
Web Attack - XSS	An attack in which malicious scripts are injected into trusted websites, by exploiting their input validation vulnerability, and executed by other end users' browsers, which have no way to distinguish it from legitimate content.
Web Attack - Sql Injection	An attack in which the attacker interferes with the queries that an application makes to its database enabling him to view data that he is not supposed to have access to, including data belonging to other users and any other data that the platform has access to, as well as modify or delete it. It is carried out by injecting malicious SQL queries into an input field, which the vulnerable application passes directly to the database as a query instead of treating it as plain data. The database then executes it, allowing the attacker to manipulate the database in unintended ways.

Infiltration	A multi-stage attack to gain unauthorized access to systems, in which the attacker first deploys a malicious file disguised as a legitimate one, such as a document or a downloadable link, containing a backdoor. The victim executes the file unknowingly, after which the backdoor establishes a reverse connection back to the attacker's machine, granting the attacker remote control over the compromised system. Once inside, the attacker scans the victim's internal network using tools (e.g. Nmap) to discover live hosts (any active and reachable device on the network), as well as open ports and running services. This allows the attacker to map the internal network and identify potential targets for further infiltration. The attacker then moves laterally by infiltrating another machine, spreading the attack further across the network. Finally, an attempt is made to exfiltrate sensitive data (transferring confidential information out of the network), and to escalate privileges on the compromised systems in order to gain access to even more restricted and sensitive resources.
Bot	An attack carried out using the ARES botnet, a malware-based network of compromised computers built on a RAT (Remote Access Trojan) and botnet framework, which allows the attacker to take control of infected machines and coordinate them for malicious purposes. It first infects victim machines using phishing emails or malicious downloads, after which the infected machines connect back to the attacker's Command and Control (C&C) server. Finally, the attacker issues commands to all bots, that is, all infected machines, simultaneously.
DDoS	An attack used to overwhelm the target system with a flood of traffic from multiple sources, making it unable to respond to legitimate requests, potentially leading to a complete shutdown of the system.
PortScan	An attack used to discover open ports and whether they are receiving or sending data, in addition to potential vulnerabilities that could be exploited.

4.3.3 Data Exploration

Table 4.7 provides a comprehensive overview of the classes' distribution across the dataset.

TABLE 4.7 – Class Distribution

Class	file	Count	Percentage
BENIGN	Monday	529,918	18.72%
	Tuesday	432,074	15.26%
	Wednesday	440,031	15.54%
	Thursday AM	168,186	5.94%
	Thursday PM	288,566	10.19%
	Friday AM	189,067	6.68%
	Friday PM (DDoS)	97,718	3.45%
	Friday PM (PortScan)	127,537	4.51%
	Total	2,273,097	80.3%
DoS Hulk	Wednesday	231,073	8.163%
PortScan	Friday PM (PortScan)	158,930	5.614%
DDoS	Friday PM (DDoS)	128,027	4.523%
DoS GoldenEye	Wednesday	10,293	0.364%
FTP-Patator	Tuesday	7,938	0.280%
SSH-Patator	Tuesday	5,897	0.208%
DoS slowloris	Wednesday	5,796	0.205%
DoS Slowhttptest	Wednesday	5,499	0.194%
Bot	Friday AM	1,966	0.069%
Web Attack - Brute Force	Thursday AM	1,507	0.053%
Web Attack - XSS	Thursday AM	652	0.023%
Infiltration	Thursday PM	36	0.0013%
Web Attack - Sql Injection	Thursday AM	21	0.0007%
Heartbleed	Wednesday	11	0.0004%
Total		2,830,743	100%

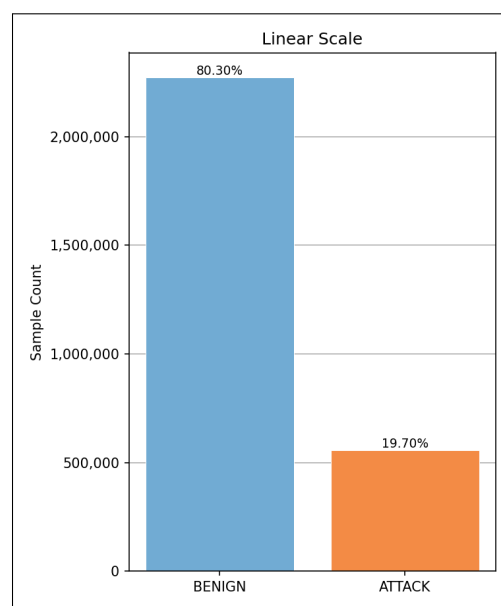


FIGURE 4.2 – BENIGN vs Attack Ratio

Figure 4.2 visualizes the BENIGN vs Attack ratio. The BENIGN class is the dominant class in the dataset, accounting for 80.3%, while the remaining 15 attack classes collectively amount to only 19.7%.

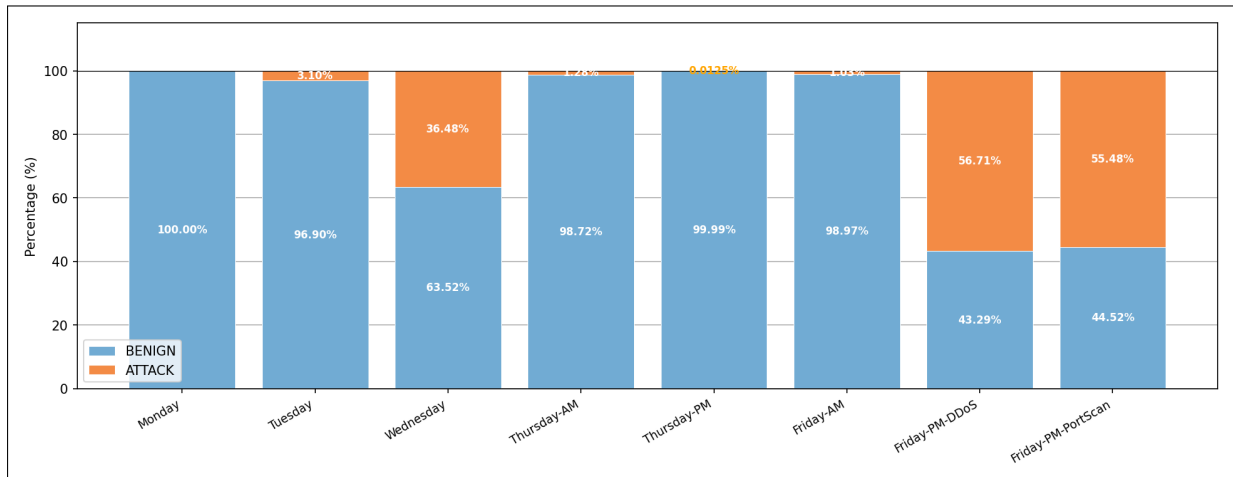


FIGURE 4.3 – Benign vs Attack Ratio per file

Figure 4.3 shows the BENIGN vs Attack ratio per file. The BENIGN class dominates across all files, with the only exceptions being the two Friday PM files, where attack traffic exceeds BENIGN, with attack ratios of 56.7% and 55.5% in the Friday PM DDoS and Friday PM PortScan respectively. The BENIGN ratio ranges from 100% in the Monday file down to 43.3% in the Friday PM DDoS file. Several files show very low attack ratios, such as Thursday AM at 1.3% and Friday AM at 1%, while the Thursday PM file has an attack ratio so low at 0.012% that it is invisible in the figure.

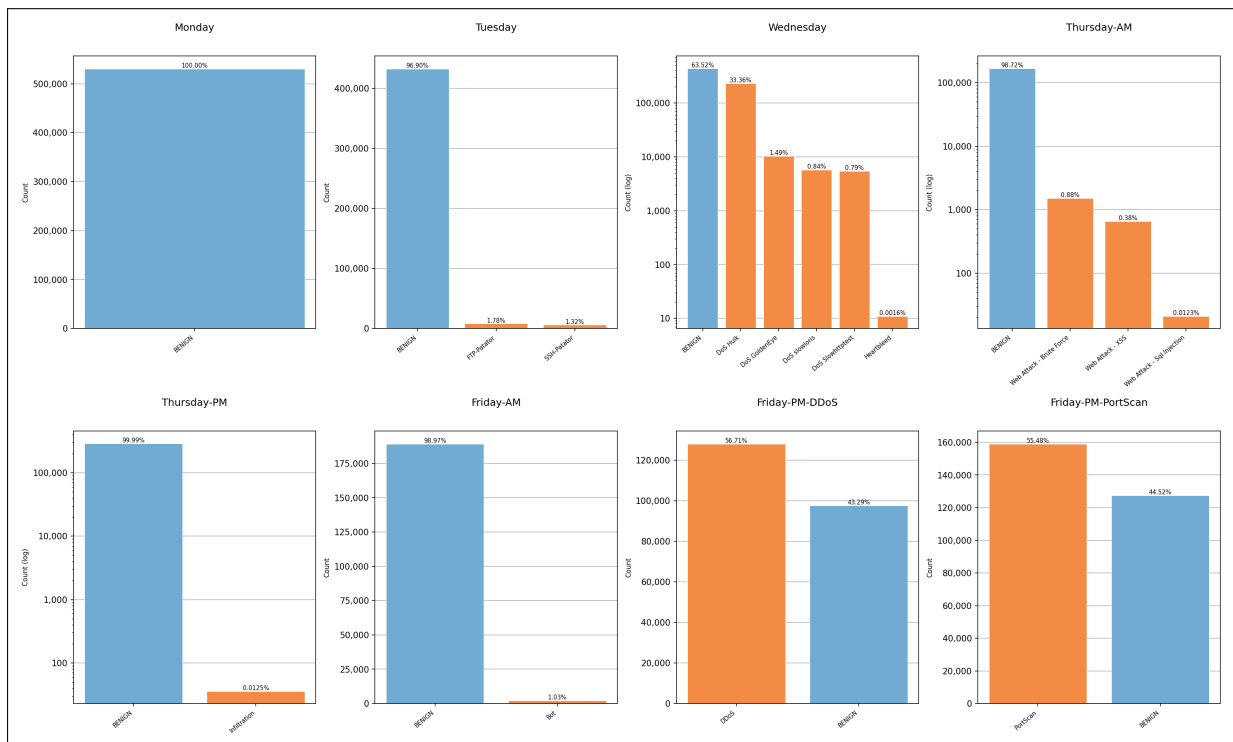


FIGURE 4.4 – Class distribution per file

Figure 4.4 visualizes the class distribution per file. As mentioned previously, each attack class is present in only one file, with each file containing between 0 and 5 attack classes, most files containing only one. The two brute force attacks, FTP-Patator and SSH-Patator are both present in the Tuesday file. The DoS attacks and Heartbleed are grouped in the Wednesday file, and the web attacks are present in the Thursday AM file. The remaining classes each appear exclusively in one of the other files.

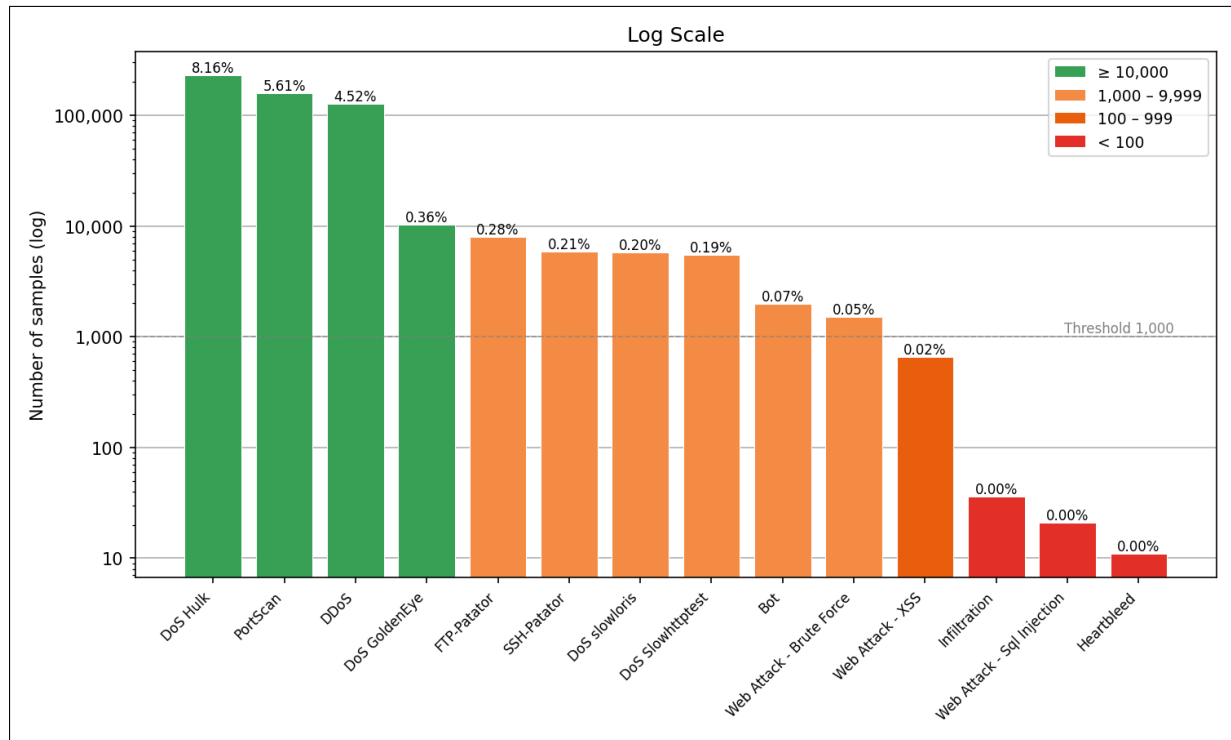


FIGURE 4.5 – Attack distribution

Figure 4.5 visualizes the global attack distribution. It varies significantly, ranging from DoS Hulk at 8.16% down to Heartbleed at 0.0004%, with the three rarest classes, Infiltration, Web Attack - Sql Injection, and Heartbleed, collectively accounting for only 0.0024%.

This distribution is intentional, as the dataset was generated to imitate real-world network traffic, where the vast majority of traffic is normal and only a small fraction are attacks. However, this introduces a severe class imbalance that can bias the model towards the majority class, leading it to treat BENIGN as the default outcome. As a result, the model may perform poorly on the attack classes and potentially fail to learn their distinguishing patterns altogether.

The class imbalance extends beyond the BENIGN class and is also present among the attack classes themselves. DoS Hulk, PortScan, DDoS and DoS GoldenEye each exceed 10,000 samples, providing the model with sufficient examples to reliably learn their patterns. FTP-Patator, SSH-Patator, DoS slowloris, DoS Slowhttptest, Bot and Web Attack - Brute Force fall in a moderate range, each with over 1,000 samples, which while on the lower end, should still allow the model to capture their general patterns.

Web Attack - XSS class, with only 652 samples sits in a more concerning range where learning becomes unreliable.

The three remaining classes represent the most critical imbalance: Infiltration with 36 samples, Web Attack - Sql Injection with 21 samples, and Heartbleed with only 11 samples. At this scale, the model is unlikely to learn their patterns meaningfully, and even in cases where it does, generalization to unseen data is very unlikely, leading to poor performance on new data.

After exploring the class distribution, we turn to the features computing the correlation matrix for the 78 features of the dataset, which is a square matrix where each element represents the correlation (linear association) between two features, with values ranging from -1 to 1 negative values means negative correlation where one feature decreases as the other increases, positive values means positive correlation where one feature increases as the other increases but both means strong correlation, and it gets stronger as it approaches 1 or -1, and values equal to zero means they have no correlation, with the diagonal being always zero as a feature is always perfectly correlated with itself.

The correlation matrix is computed using the PCC (Pearson Correlation Coefficient) also known as Pearson's r :

$$r_{xy} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (4.1)$$

where x is the first feature and y is the second.

We took the absolute value of the correlation matrix focusing on the strength of the correlation regardless of the direction. Using a threshold of 0.95 to identify highly correlated features, any feature pair having a correlation coefficient of 0.95 or higher means they essentially provide the same information (redundant features) meaning they provide no additional information to the model.

39 pairs are identified they are visualized in the correlation heatmap in Figure 4.6.

8 pairs have a perfect correlation of 1, meaning they are perfectly linearly correlated. Table 4.8 lists these pairs.

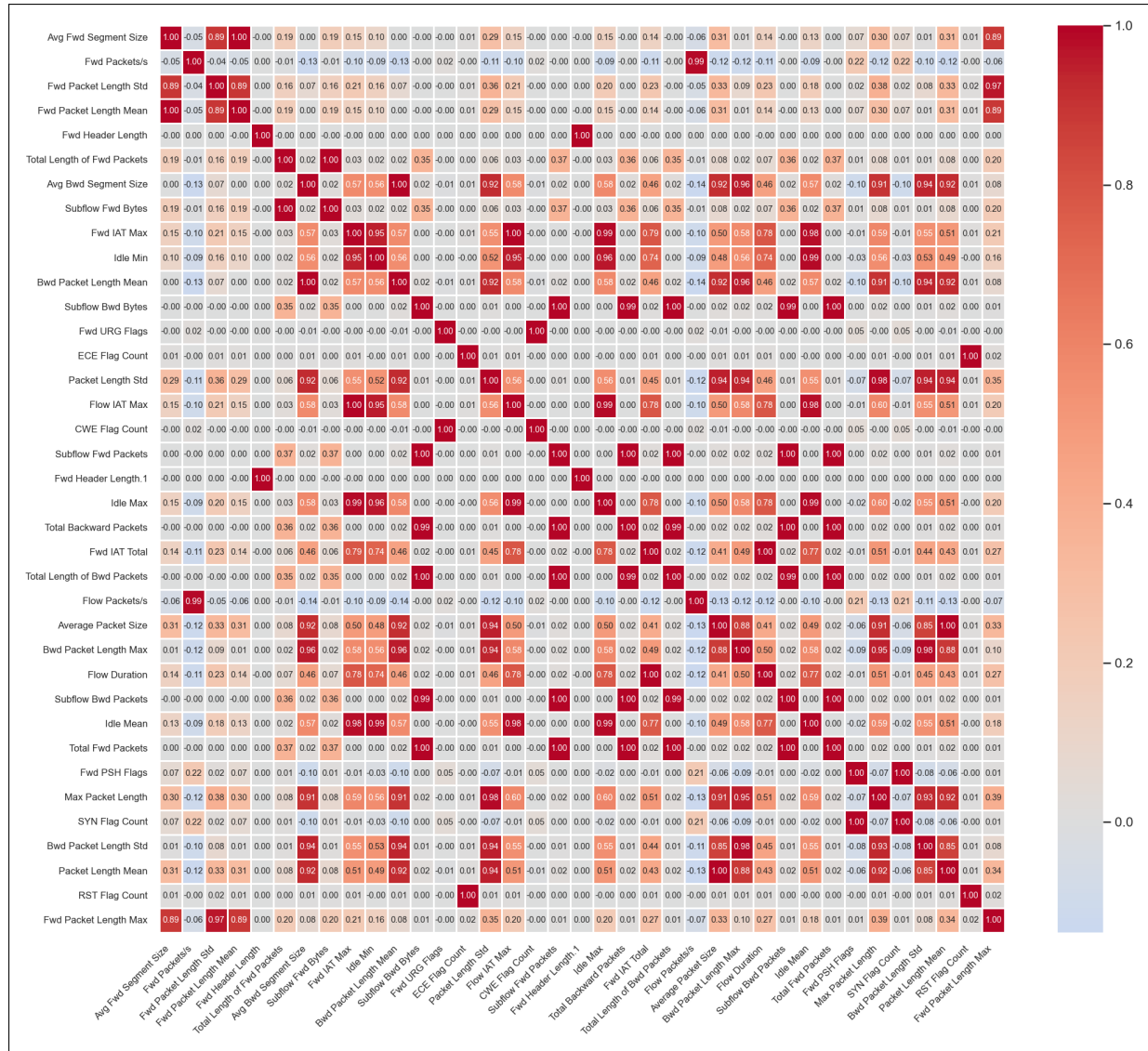


FIGURE 4.6 – Correlation Heatmap

TABLE 4.8 – Perfectly Correlated Features

No.	Feature 1	Feature 2
1	Bwd Packet Length Mean	Avg Bwd Segment Size
2	Fwd Packet Length Mean	Avg Fwd Segment Size
3	Total Fwd Packets	Subflow Fwd Packets
4	Total Backward Packets	Subflow Bwd Packets
5	Total Length of Bwd Packets	Subflow Bwd Bytes
6	Fwd PSH Flags	SYN Flag Count
7	Fwd URG Flags	CWE Flag Count
8	Fwd Header Length	Fwd Header Length.1

- **First and second pairs:** Bwd Packet Length Mean and Avg Bwd Segment Size, Fwd Packet Length Mean and Avg Fwd Segment Size going by name the two calculates the average size of two different layers of the OSI model, the former belonging to the network layer it is composed of Network Header (IP header) + Segment,

while the latter belongs to the transport layer and is composed of Transport Header (TCP/UDP Header) + Payload, but in the CICFlowMeter they are both calculated as the average size (bytes) of the payload in the backward/forward direction, which means there is no difference between a segment and a packet making the two features a duplicate of each other.

- **Third, fourth and fifth pairs (with an additional pair):** `Total Fwd Packets` and `Subflow Fwd Packets`, `Total Backward Packets` and `Subflow Bwd Packets`, `Total Length of Fwd Packets` and `Subflow Fwd Bytes`, and `Total Length of Bwd Packets` and `Subflow Bwd Bytes` (near perfect correlation of 0.99999) nominally represent two different levels of aggregation, with the first being for the entire flow and the second being for a subflow. A subflow is detected when the inter-arrival time (IAT) between two consecutive packets exceeds 1 second, with the subflow features computed by dividing the flow totals by the number of detected subflows (`sfCount`). The assumption that all subflows contain the same number of packets or bytes is already a conceptual limitation, but the way `sfCount` is calculated is a bug in itself. Due to a misplaced parenthesis in the source code, the condition that is supposed to check whether the gap between two packets exceeds 1 second fires on the first inter-packet gap that does so and never again, making `sfCount` equal to 1 for virtually every flow in the dataset. As a result, all four subflow features reduce to dividing the flow total by 1, making them equal to their corresponding totals. The packet-count pairs hold this equality perfectly throughout the entire dataset since integer division by 1 is exact. The byte pairs behave the same way for the vast majority of flows, with rare divergence explained not by `sfCount` but by a type mismatch: `Total Length of Fwd/Bwd Packets` is output as a `double`, while `Subflow Fwd/Bwd Bytes` is output as a `long`. For flows with very large byte totals, floating-point precision loss in the `double` path produces a value that differs slightly from the exact `long`, which is the only source of divergence observed. All four pairs are therefore redundant, carrying no additional information beyond their corresponding flow-level totals, a consequence of a single bug.

4.3.4 Data quality

The quality of a model’s output is directly tied to its input. In this section we evaluate the dataset for any quality issues that could disrupt model training such as missing values (NaN - Not a number), infinite values (inf), negative values, duplicates rows, constant and near-constant features. For each issue, the analysis investigates the root cause, assesses its global impact on the dataset, across classes and whether it endangers any minority classes.

NaN and Infinite Values: Scanning the dataset for undefined values revealed that NaN values appear exclusively in `Flow Bytes/s`, while infinite values appear in both `Flow Bytes/s` and `Flow Packets/s`. To understand the origin, the CICFlowMeter source code

was inspected directly. The relevant formulas are:

$$\text{Flow Bytes/s} = \frac{\text{forwardBytes} + \text{backwardBytes}}{\text{flowDuration}/1,000,000}$$

$$\text{Flow Packets/s} = \frac{\text{packetCount (fwd + bwd)}}{\text{flowDuration}/1,000,000}$$

Both features divide by **Flow Duration**. When **Flow Duration** = 0, any nonzero numerator yields Inf, and a zero numerator yields NaN 0/0. This distinction maps directly onto two observable row types in the dataset:

- Rows with 2 forward packets, 0 backward, and nonzero byte count:
Flow Duration = 0 with bytes > 0, producing **Flow Bytes/s** = Inf.
- Rows with 1 forward packet, 1 backward packet, and zero byte count:
Flow Duration = 0 with zero bytes, producing **Flow Packets/s** = NaN.

The zero byte count in the second case, despite existing packets, is because CFlowMeter computes **Total Length of Fwd/Bwd Packets** from the packet payload only. Packets carrying no data content only TCP headers (ACK, SYN) produce zero payload length. The true anomaly in both cases is **Flow Duration** = 0.

Flow Duration is computed as the timestamp difference between the last and first packet in a flow. It evaluates to zero when two packets arrive within the same timestamp resolution window as the tool lacks the granularity to distinguish arrivals that are nanoseconds apart. Every affected row contains exactly two packets total, either two forward or one forward and one backward, which is consistent with this explanation: with only two packets, a single resolution failure collapses the entire duration to zero. This collapse propagates to every other feature that depends on the timestamp value, like **Flow IAT Mean**, **Flow IAT Max**, **Flow IAT Min**, and all rate-based features reduce to zero as well.

These rows impact is as follows. Globally, 353 unique row carry at least one NaN value making it 0.01% of the dataset, and 1,564 unique row carry at least one Inf value making it 0.05% of the dataset. At class level, Inf impact BENIGN with 1,427 unique row making it 0.06% of the class samples and only four attack classes are affected. NaN infects only BENIGN with 350 making it less than 0.01% of the class samples along with Dos Hulk. Rare classes such as Heartbleed, Infiltration, and Web Attack – SQL Injection are not affected. The attack class-level distribution is shown in the 4.7 Figure.

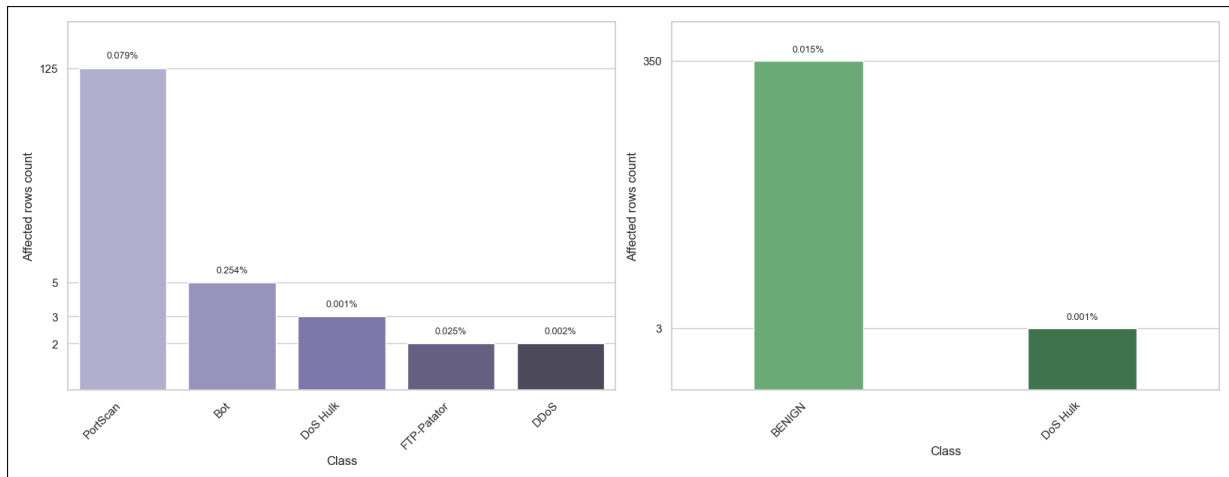


FIGURE 4.7 – Inf and NaN attack class distribution

Negative Values: Given the nature of the dataset feature, their logical value should never be negative yet Negative values were found across several feature groups, each with a distinct cause.

- **Sentinel encoding:** `Init_Win_bytes_forward` and `Init_Win_bytes_backward`.

These two features carry negative values at scale: 1,001,189 rows (35.4%) and 1,441,552 rows (50.9%) respectively hold the value -1. This is not a measurement error. CICFlowMeter uses -1 as a sentinel to signal that the TCP initial window size could not be captured, either because the SYN packet was absent from the capture, or because the flow is UDP-based and has no TCP handshake. It is a deliberate encoding choice by the tool, not a data corruption.

- **Packet ordering artifacts:** `Flow Duration`, `Flow IAT Mean`, `Flow IAT Max`, `Flow IAT Min`, `Fwd IAT Min`, `Flow Bytes/s`, and `Flow Packets/s`.

A total of 115 rows carry negative values in `Flow Duration` ranging from -13 to -1, with the same negative values propagating identically into `Flow IAT Mean`, `Flow IAT Max`, and `Flow IAT Min`.

This is a cascade, since each of these rows has exactly one packet forward and one backward, there is only one inter-arrival interval in the flow, so mean, max, and min all equal `Flow Duration` exactly.

Division by a small negative duration scaled by 1,000,000 then causes `Flow Bytes/s` and `Flow Packets/s` to carry large negative values.

For example, with `Flow Duration` = -1 and 12 bytes of payload:

$$\text{Flow Bytes/s} = \frac{12}{-1/1,000,000} = -12,000,000$$

This is likely to be a packet ordering artifact when processing the packets; additionally, CICFlowMeter performs no guard against negative duration, so the error propagates unchecked. The affected rows are exclusively BENIGN traffic and represent less than 0.0001% of the dataset.

After excluding these 115 rows, an additional 2,776 rows exhibit negative Flow IAT Min values and 17 rows exhibit a negative Fwd IAT Min. Here the ordering artifact affects only a single packet pair within a longer multi-packet flow, dragging the minimum IAT negative while all other temporal features remain valid. The attack class-level distribution is shown in the 4.8 Figure.

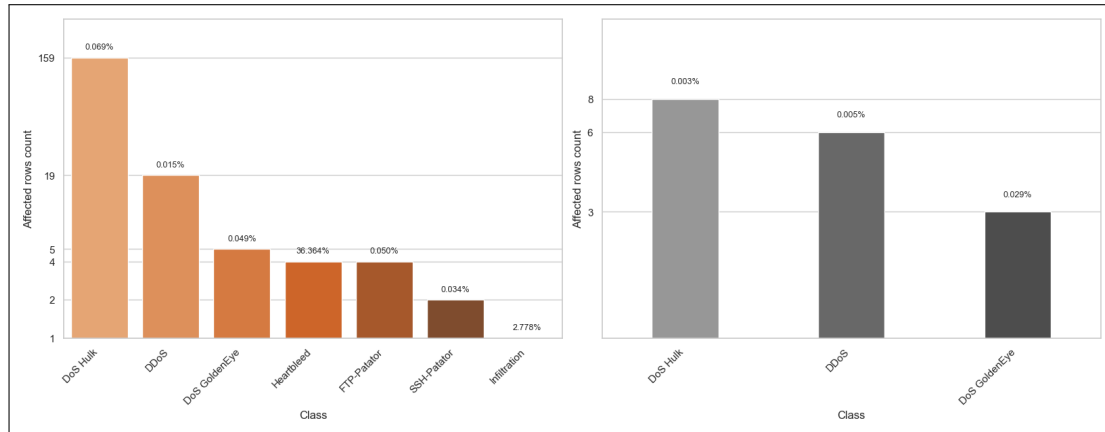


FIGURE 4.8 – Ordering Artifact Class Distribution

- **UDP parsing bug:** Fwd Header Length, Fwd Header Length.1, Bwd Header Length, min_seg_size_forward.

These features exhibit values in the range of -32,212,234,632 to -1,073,741,320, far outside any legitimate segment size and a TCP header size, which is bounded between 20 and 60 bytes.

The root cause is a bug in CICFlowMeter: when processing UDP packets, the tool reads the header size from a TCP-specific field that simply does not apply to UDP, producing meaningless values that corrupt these features.

The bug is confirmed by the data: all 35 affected rows are UDP flows, as indicated by `Init_Win_bytes_forward = Init_Win_bytes_backward = -1`.

The impact is narrow, 35 rows in Fwd Header Length, Fwd Header Length.1, and min_seg_size_forward, and 22 rows in Bwd Header Length, all exclusively BENIGN traffic, collectively under 0.0001% of the dataset.

Exact duplicates: Duplicate rows are common in network datasets since a row in the dataset does not represent a packet but a session behaviour summarised through statistical aggregation. Many flows share identical statistical profiles, thus the presence of duplicates does not necessarily indicate a data error.

Globally, duplicate rows represent 308,381 out of 2,830,743 rows (10.89%). Their distribution by attack class is as show in the 4.9 figure and no minority class is affected.

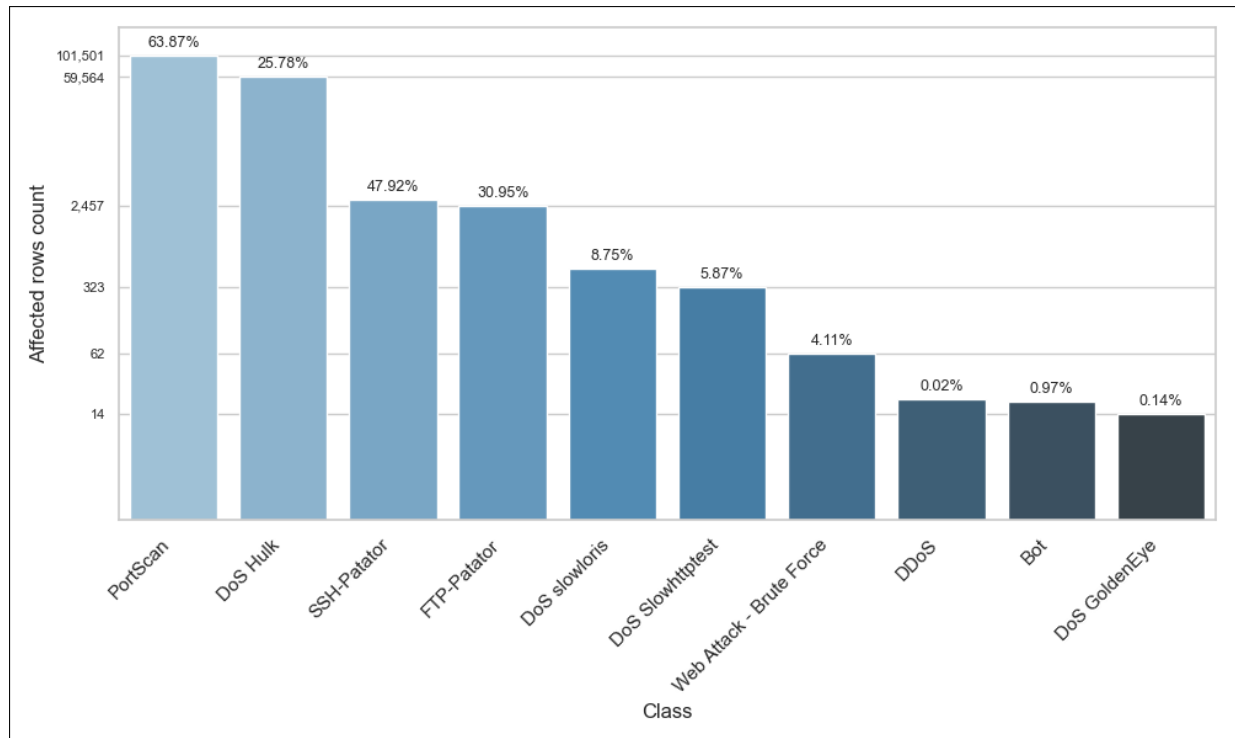


FIGURE 4.9 – Exact duplicates attack class distribution

Contradictory duplicates: These rows share the exact same feature values except for the target column, their labels differ. CICFlowMeter aggregates raw packet-level captures into flow statistics. Two flows from different sessions, one BENIGN and one malicious, can produce statistically identical feature vectors if their traffic patterns are indistinguishable at the flow-aggregation level. This is a fundamental limitation of statistical flow-based features rather than a labelling error.

Globally, 7,020 rows are affected. BENIGN traffic is the dominant participant in contradictory groups, as expected, BENIGN flows vastly outnumber attack flows, so the probability of a BENIGN flow sharing a feature vector with an attack flow is higher than inter-attack collisions. Contradictions involve only well-represented classes: DoS Hulk, DDoS, DoS slowloris, and PortScan. The 4.10 figure illustrates their distribution.

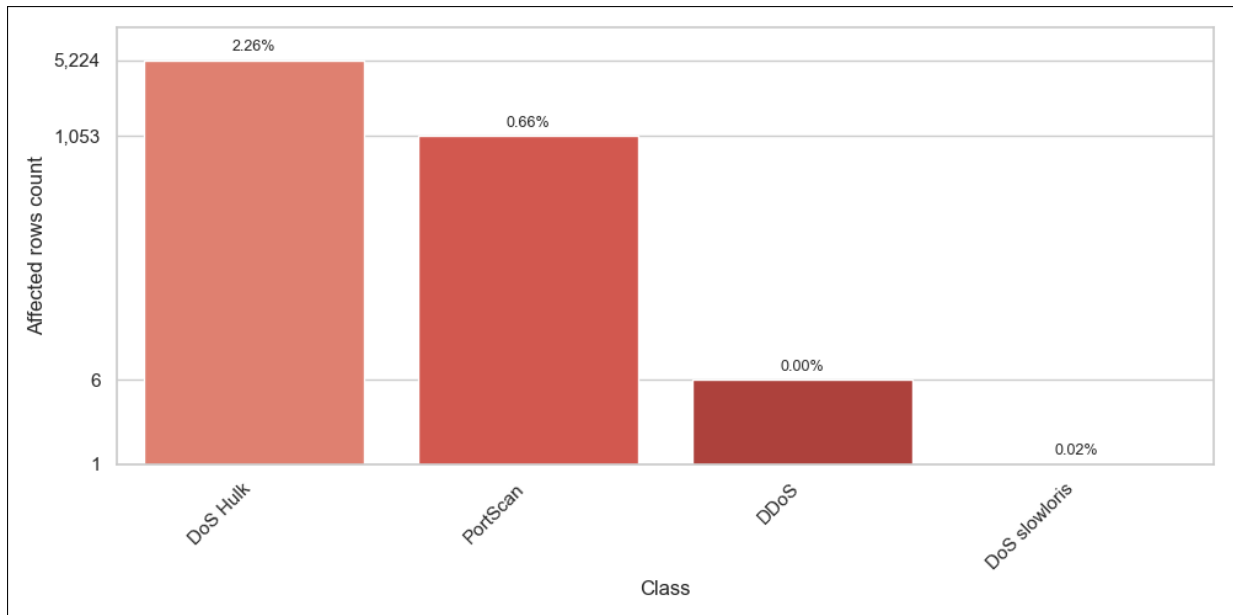


FIGURE 4.10 – onrtradiictory duplicates attack class distribution

Constant features: Constant features are those with a single unique value across the entire dataset, in this case, always zero. To understand this behaviour, each feature was traced back to its computation logic in the CICFlowMeter source code. The root cause divides them into two groups.

- **Bulk transfer features:** Fwd Avg Bytes/Bulk, Fwd Avg Packets/Bulk, Fwd Avg Bulk Rate, Bwd Avg Bytes/Bulk, Bwd Avg Packets/Bulk, Bwd Avg Bulk Rate.

Bulk transfer refers to sending a large number of packets consecutively in one direction, with very little time between them.

These features are derived from CICFlowMeter’s bulk detection logic, which only activates when at least four consecutive packets carrying actual data are observed in close succession.

If that condition is never met, either because flows in this dataset rarely involve this kind of continuous data sending, or because many packets carry no payload, the counters stay at zero and all six features are exported as zeros.

This is exactly what is observed. These features are constant not by error, but because the bulk detection logic simply never triggers on this dataset.

- **Backward flag counters:** Bwd PSH Flags and Bwd URG Flags

These are constant zero due to a bug in CICFlowMeter’s implementation. The features Bwd PSH Flags and Bwd URG Flags are meant to count how many times the PSH and URG flags appear in packets traveling in the backward direction.

However, these counters are only incremented inside the method that processes the first packet. Since the first packet by definition can only be forward, CICFlowMeter assigns the source IP from that first packet and compares subsequent packets against it to determine direction. The method that handles the following packets never

updates these counters, so both features remain zero for every flow, regardless of the actual traffic.

Near-constant features: Fwd URG Flags, CWE Flag Count, RST Flag Count, and ECE Flag Count.

Each feature hold exactly two unique values across the entire dataset. These are rare-event TCP control flags that are virtually absent in normal traffic. Combined, their non-zero occurrences account for only 315 rows (0.0139%), all within BENIGN traffic. Therefore, they carry no discriminative signal for any attack class.

4.4 Data preparation

4.4.1 Data cleaning

The integrity of data is a prerequisite for any reliable model. Throughout this section, data correctness is prioritized over sample retention, as long as data volume and class balance are not meaningfully affected.

NaN and Inf Values: Rows carrying NaN or Inf values in **Flow Bytes/s** and **Flow Packets/s** are removed. As established, these arise from division by a zero **Flow Duration**, due to the lack of time granularity. Beyond the invalid rate features, these rows are also dominated by zeros which do not reflect the true nature of the flow, but are simply an artifact of insufficient measurement precision. Such a row cannot serve as a valid representation of a network session, as its defining characteristics were lost in the granularity gap.

The scale reinforces the decision. Globally, fewer than 0.11% of rows are affected. Additionally, the unique affected rows per attack class amount to 5 for Bot, 125 for PortScan, 2 for DDoS, and 3 for DoS Hulk, representing less than 0.00% of any single class, all of which carry sufficient clean samples to generalise from. No minority class is affected.

Negative Values: Negative values across the dataset fall into four distinct categories, each handled differently.

- **Sentinel encoding:** `Init_Win_bytes_forward` and `Init_Win_bytes_backward`.

These are retained without modification. As noted, -1 is a deliberate sentinel used by CICFlowMeter to encode the absence of a TCP handshake, it is semantically valid and carries information about the flow type. Replacing it would discard a meaningful signal.

- **Packet ordering artifacts:** This anomaly originates during packet processing, since these sessions consist of a single packet exchange, the flow has no other packets to establish a valid duration or inter-arrival times. **Flow Duration** was rendered

negative, and all rate-based features, `Flow IAT Mean`, `Flow IAT Max`, and `Flow IAT Min` collapsed to that same negative value, leaving no part of the flow intact.

Beyond the specific features where the corruption is visible, there is a deeper concern. This is a bug introduced during dataset construction, and its effects are not fully observable from the CSV alone.

Negative values in `Flow Duration` make the anomaly detectable, but there is no guarantee that the ordering artifact confined its damage only to features that produce obviously invalid numbers.

The same error may have silently distorted other features into values that appear plausible but are incorrect, damage that is invisible precisely because it does not manifest as negatives or zeros, but as ordinary-looking numbers that simply do not reflect the true flow.

In order to preserve data integrity and given the rows effect is negligible, only 115 rows belonging to BENIGN traffic, we decide to drop these rows.

`Flow IAT Min` and `Fwd IAT Min` share the same root cause, a packet ordering artifact, but the outcome is fundamentally different. Here the affected flows span multiple packets, so the ordering error disturbed only a single packet pair within an otherwise well-measured session.

The negative value appears only in the minimum IAT statistic, while `Flow Duration` remains positive and all other features are valid.

Inter-arrival time cannot be negative by definition, so clipping both features to zero restores the value to the physical boundary it should never have crossed, without fabricating anything.

This targeted correction is justified precisely because the flow itself was captured correctly, unlike the single-packet cases above, where the corruption was total and removal was the only option.

This treatment also preserves the 4 Heartbleed rows that would otherwise be discarded, a meaningful consideration given the class contains only 11 samples in total.

- **UDP parsing bug:** `Fwd Header Length`, `Bwd Header Length`, `Fwd Header Length.1`, and `min_seg.size_forward`.

These rows are removed. As established, they are caused by a parsing bug rendering these feature values meaningless and irrecoverable. The fact that all affected rows belong exclusively to BENIGN traffic and represent under 0.002% of the dataset confirms that removal incurs no meaningful cost to class balance or dataset integrity.

Exact duplicates: Exact duplicate rows are dropped. Retaining them risks data leakage, a row could appear in both the training and test sets simultaneously, moreover it risks overfitting by biasing the model toward a single flow pattern. Since no minority classes are affected, the removal carries no risk to rare attack samples.

Contradictory duplicates: These rows present a serious risk to model training. An identical feature vector mapped to more than one target is unresolvable, retaining such rows forces the model to fit contradictory labels from the same input. All rows belonging to contradictory groups are therefore dropped entirely, regardless of their label. Since rare classes are unaffected, this carries no risk of losing minority attack samples.

4.4.2 Feature engineering

The correlation matrix is recomputed on cleaned data as the composition of the dataset has changed. 13 new pairs emerge and 3 pairs no longer appear. In total, 49 pairs of highly correlated features are identified they provide no additional information to the model, the correlation between some of them is caused by bugs in the computation of the features.

The absence of the 3 pairs is explained by the cleaning process: **Fwd URG Flags** and **CWE Flag Count**, **RST Flag Count** and **ECE Flag Count** are near-constant features, **Fwd Header Length** and **Fwd Header Length.1** are duplicate columns. As these features are no longer present in the dataset, the correlated pairs they formed are no longer present in the recomputed correlation analysis.

For the 13 new pairs, each pair has at least one of these two features: **Fwd Header Length**, **Bwd Header Length**. These two features have their UDP parsing-corrupted rows dropped. Since the correlation analysis is recomputed on the cleaned dataset, the true underlying correlation between these features and others becomes visible, previously masked by the corrupted values.

One feature from each pair is dropped, the choice is made based on Mutual information (MI score) calculating the feature-target association to determine which of the features is more useful to distinguish the classes, then knowledge based decision for the pairs that has similar MI score.

Mutual Information (MI) measures how much knowing the value of a feature reduces uncertainty about the target class. It captures non-linear relationships as well as linear ones, it works on categorical and continuous features. If a feature has 0 MI score, it means it provides no information aiding in the target prediction, while the higher the MI score, the more useful the feature is for class distinction. The MI score tells the relationship between single feature and the target independently of other features:

$$MI(X;Y) = \sum_{x \in X} \sum_{y \in Y} p(x,y) \log \frac{p(x,y)}{p(x)p(y)} \quad (4.2)$$

Where X is the feature and Y is the target, $p(x,y)$ is the joint probability distribution of X and Y , and $p(x)$ and $p(y)$ are the marginal probability of X and Y independently.

The decision for 37 pairs of the 49 highly correlated pairs, the feature having the higher MI score is retained, however those decisions may be overridden by the Knowledge based decision.

Looking at what each feature represents in terms of network behavior rather than its statistical properties the decision for the remaining 12 pairs is made. Table 4.9 features the decision for these pairs.

TABLE 4.9 – Domain Knowledge decision

No	Feature 1	Feature 2	Decision and Justification
1	Total Fwd Packets	Subflow Fwd Packets	Total Fwd Packets is kept as it
2	Fwd PSH Flags	SYN Flag Count	SYN Flag Count is kept as it
3	Fwd Packet Length Mean	Avg Fwd Segment Size	Fwd Packet Length Mean is kept
4	Total Backward Packets	Subflow Bwd Packets	Total Backward Packets is kept for the same reason as the first pair the sole difference is that the first pair is for forward packets and this pair is for backward packets.
5	Bwd Packet Length Mean	Avg Bwd Segment Size	Bwd Packet Length Mean is kept for the same reason as the third pair the sole difference here too is the direction of the packet the other pair is forward while this one is backward.
6	Total Length of Bwd Packets	Subflow Bwd Bytes	Total Length of Bwd Packets is kept
7	Total Length of Fwd Packets	Subflow Fwd Bytes	Total Length of Fwd Packets is kept for the same reason as the sixth pair and the sole difference again is the direction of the packet.
8	Flow Duration	Fwd IAT Total	Flow Duration is kept as it is a direct and universally understood measure of how long the connection lasted while Fwd IAT Total is the total inter-arrival times between consecutive forward packets, a derived aggregate of the Flow Duration that is harder to interpret directly.
9	Flow Packets/s	Fwd Packets/s	Flow Packets/s is kept as it is more complete than Fwd Packets/s , the forward direction rate of the flow. It is the total packet rate of the flow in both direction combined, it is more likely to capture attack patterns since attacks can manifest as abnormal rates in either or both direction.
10	Idle Mean	Idle Max	Idle Mean is kept as it is more stable and representative of typical idle (inactive state) behavior, it is the average idle period within a flow while the Idle Max is the longest idle period, that a single unusually long pause can easily inflate it without reflecting the overall pattern.
11	Idle Mean	Idle Min	Idle Mean is kept as Idle Min captures the minimum pause that is easily influenced by noise and again Idle Mean is more informative as it summarizes the typical idle behavior across the flow.

12	Idle Max	Idle Min	Both are dropped in favor of Idle Mean , as they represent extremes of the idle time distribution. While an unusually long idle period may occasionally reflect meaningful behavior, an unusually short one is easily influenced by noise. Neither is as representative of typical idle behavior as Idle Mean .
----	----------	----------	--

In conclusion, of the 33 features involved in the highly correlated pairs, 19 are dropped:

- Avg Fwd Segment Size
- Bwd Header Length
- Bwd Packet Length Max
- Bwd Packet Length Std
- Flow IAT Max
- Fwd Header Length
- Fwd IAT Total
- Fwd PSH Flags
- Fwd Packet Length Std
- Fwd Packets/s
- Idle Max
- Idle Min
- Max Packet Length
- Packet Length Mean
- Subflow Bwd Bytes
- Subflow Bwd Packets
- Subflow Fwd Bytes
- Subflow Fwd Packets

4.4.3 Label encoding

The selected model, XGBoost, requires numerical input for both features and the target variable. While all features are already numerical, the target column **Label** is categorical and stored as an object type. Hence, Label Encoding was performed using Label encoder from `sklearn.preprocessing`. The encoder was fitted and applied to the target column in a single step using `fit_transform()`. To preserve interpretability of the model's predicted outputs, the resulting integer-to-class mapping was exported as a `label_mapping.json` file derived from `le.classes_`, allowing any predicted label to be traced back to its original class name.

4.4.4 Train/ Test split

Several factors make us approach this step with caution, when assessing the train/test split strategies.

A per-file split is not an option, as the files are divided per attack type, each attack class appears in exactly one file, so splitting by file would result in classes entirely absent from either the training or test set.

A standard random 80/20 split presents its own issues. The CIC-IDS2017 dataset is heavily imbalanced, the BENIGN to attack ratio is 80/20. Additionally, rare classes present less than 0.001% of the full dataset, so this strategy offers no guarantee of a fair

class distribution and gives no assurance that rare classes will be adequately represented in both sets, which is particularly concerning given their scarcity.

For these reasons, a stratified train/test split is selected. It ensures all classes are present in both the training and test sets, each maintaining their original proportions, so the class distribution in each split mirrors that of the full dataset.

However, this strategy doesn't solve the class imbalance. This issue will be addressed and treated more during the model training.

4.5 Conclusion

Chapter 5

Modeling

5.1 Introduction

5.2 State of the Art

5.3 Adopted Approach

5.4 Training and Hyperparameter Optimization

5.5 Conclusion

Chapter 6

Realization

6.1 Introduction

6.2 Development Tools and Technologies

6.3 Performance Analysis and Evaluation Metrics

6.4 User Interfaces

6.5 Integration

6.6 Conclusion

General Conclusion

Webography

- [1] <https://www.snort.org/>. Accessed on: April 6, 2026.
- [2] <https://suricata.io/>. Accessed on: April 6, 2026.
- [3] <https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-introduction/>. Accessed on: April 2, 2026.
- [4] <https://github.com/ahlashkari/CICFlowMeter>. Accessed on: April 23, 2026.

Bibliography

- [1] Chris Jackson. *Network Security Auditing*. 800 East 96th Street, Indianapolis, IN 46240 USA: Cisco Press, 2010. ISBN: 978-1-58705-352-8.